Leetcode 35 :- Search Insert Position →

↳ O(n)

([1, 3, 5, 6], target = 2)

i=0 i=1 i=2 i=3

target = 4

target = 7

1 <= 2 → ans = i+1

3 <= 2 ✗

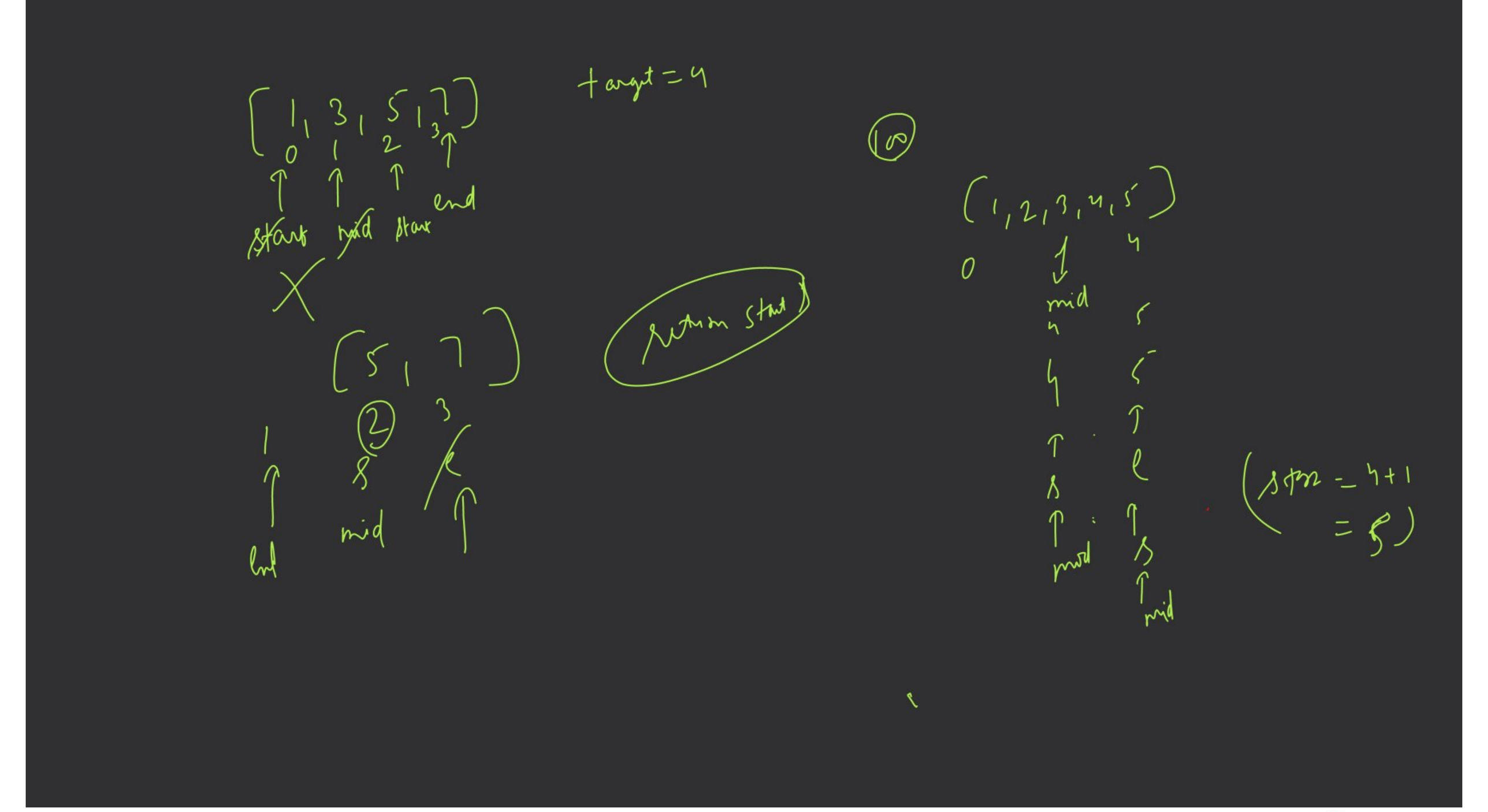
```
for(i=0; i<n; i++){
    if(nums[i] >= target)
        return i;
}
return nums.length;
```

Or

```
if(nums[i] < target)
    ans = i+1;
return ans;
```

```
if(nums[i] == target)
    return i;
```

(2) (1e) (10)


[1, 3, 5, 7]
target = 4
0 1 2 3
start mid start end
[5, 7]
return start
(1, 2, 3, 4, 5)
mid
mid
mid
(start = 4+1 = 5)

[10, 20, 30, 40, 50], target = 30, 45, 60

0 1 2 3 4

s ↑ e

start = 0, end = nums.length - 1;

(start + end) / 2

```
while (start <= end) {
  int mid = (start + end) / 2;
  if (nums[mid] == target)
    return mid;
  else if (nums[mid] > target)
    end = mid - 1;
  else
    start = mid + 1;
}
return start;
}
```

size = nums.length;

int first = firstOccurrence(nums, size, target);

int last =

return new int[]{first, last};

[2, 3, 3, 5, 5, 5, 5, 5, 5, 5, 7, 7, 7, 7, 8, 8]
 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

start, mid, end

→ ans = mid

end = mid - 1

int[] arr = {1,2,3};

int[] arr;

arr = new int[]{1,2,3};

for(i = 0 → n){

if(arr[i] == target)

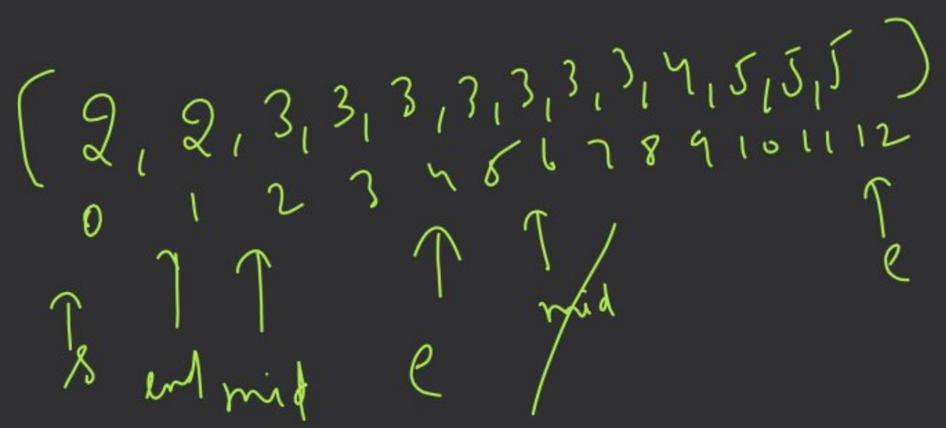
last = i

int[] arr = {first, last};

return arr;

public int firstOccurrence(int[] nums, int st, int target){

return first;




```
if(arr(mid)==target){
    ans = mid
    end = mid-1;
}
else if(arr(mid)>target){
    end = mid-1;
}
else{
    start = mid+1;
}
```

ans = 69

Problem List

Submit

1:1 Sessions with me

90

00:00:00

Premium

Code

Java

Auto

```java
import java.util.*;

class Solution {
    public int firstOccurrence(int []nums,int size,int target){
        int start = 0, end = size - 1;
        int ans = -1;

        while(start <= end){
            int mid = (start + end) / 2;
            if(nums[mid] == target ){
                ans = mid;
                end = mid - 1;
            }else if(nums[mid] > target) end = mid - 1;

            else start = mid + 1;
        }
        return ans;
    }


    public int lastOccurrence(int []nums,int size,int target){
        int start = 0, end = size - 1;
        int ans = -1;

        while(start <= end){
             int mid = (start + end) / 2;
            if(nums[mid] == target ){
                ans = mid;
                start = mid + 1;
            }else if(nums[mid] > target) end = mid - 1;

            else start = mid + 1;
        }
        return ans;
    }

    public int[] searchRange(int[] nums, int target) {
        int size = nums.length;
```